

Memorandum on the EasyCrypt Formalization of the HAETAE NTT

Codex Proof Report

22 April 2026

Abstract

This note records the present state of the EasyCrypt proof development for the HAETAE number-theoretic transform, with particular attention to the refinement path from the extracted Jasmin program to the abstract polynomial specification. The report is written in the style of a mathematical proof memorandum: completed lemmas are stated by role, partially reconstructed arguments are isolated precisely, and the remaining obligations are organized according to the logic of the final correctness theorem. The principal technical achievements to date are the proof of losslessness for the imperative NTT specifications, the reconstruction of the root-of-unity identity $zroot^{256} = -1$ in $\mathbf{F}_q$, and the establishment of a substantial portion of the representation layer relating machine words and mathematical coefficients. The main unresolved point at present is a library-level normalization issue in the concrete twiddle-table bridge, specifically around the rewriting of `BArray1024.get32_of_list32` inside the full theory `NTT_Fq.ec`.

1 Objective and Verification Architecture

The final theorem sought in the project is an end-to-end refinement statement: the Jasmin routines implementing the forward and inverse NTT must coincide, under a suitable representation relation, with the mathematical transformations defined over the quotient ring formalized in `Rq.ec`. In conceptual terms, the proof must traverse four layers:

Jasmin source $\implies$ EasyCrypt extraction $\implies$ imperative EasyCrypt NTT model $\implies$ abstract ring-theoretic N

The relevant artifacts are as follows.

- `extract/Hpoly_extract.ec`: extracted Jasmin code, including the concrete tables `jzetas` and `jzetas_inv`.
- `NTT_Fq.ec`: imperative NTT specifications together with the algebraic and representation lemmas required to relate program state to mathematical state.
- `Fq.ec` and `Montgomery.ec`: arithmetic correctness of modular and Montgomery operations.
- `Rq.ec`: abstract forward and inverse NTT operators.

2 Foundational Proofs Already Established

2.1 Losslessness of the Imperative Specifications

The imperative module `NTT` in `NTT_Fq.ec` contains two procedures:

- `NTT.ntt`, modeling the forward transform with eight butterfly layers;
- `NTT.invntt`, modeling the inverse transform with eight inverse layers followed by the final scaling by `inv(256)`.

Both procedures are now accompanied by complete losslessness proofs:

```
lemma ntt_spec_ll : islossless NTT.ntt,
lemma invntt_spec_ll : islossless NTT.invntt.
```

These proofs proceed by explicit integer variants for the three nested loops. For the forward transform one uses:

- outer variant: `len`;
- middle variant: `256 - start`;
- inner variant: `start + len - j`.

For the inverse transform the final scaling loop contributes the additional variant `256 - j`, while the outer loop uses `256 - len`.

The supporting lemmas

```
ntt_middle_right,    inv_middle_right,    inv_outer_progress,    ntt_init_progress
```

encode the required arithmetic monotonicity of these variants. Their role is entirely proof-theoretic: they guarantee that the imperative models are legitimate targets for subsequent pRHL and Hoare refinements.

2.2 Root-of-Unity Arithmetic

The HAETAE parameters are fixed by

$$q = 64513, \quad N = 256, \quad zroot = 426 \in \mathbf{F}_q.$$

The crucial algebraic fact needed for the inverse transform is now proved:

```
lemma exp_zroot_256 : Zq.exp zroot 256 = incoeff (-1).
```

The argument is economical. One first identifies

$$zroot^{128} = 28837$$

by reading the value at index 1 in the explicit forward zeta table, and then derives

$$zroot^{256} = (zroot^{128})^2 = -1 \pmod{q}.$$

This confirms that `zroot` behaves as a primitive 512-th root of unity, which is precisely the structural fact needed to justify the inverse normalization.

3 Representation Layer Between Machine Words and Coefficients

The extracted Jasmin code operates on byte arrays of packed 32-bit words, whereas the mathematical specification is expressed over arrays of field coefficients. The present development already contains the nucleus of the representation relation:

```
word_to_coeff(w) = incoeff(W32.to_sint w),
barray256_to_poly(rp) = Array256.init(λi. word_to_coeff(BArray1024.get32 rp i)),
poly_repr(rp, p) ≡ barray256_to_poly(rp) = p.
```

This layer is supported by several useful conversion lemmas:

- `barray256_to_polyE`;
- `poly_repr_get`;
- `word_to_coeff_of_int`;
- `word_to_coeff_small`;
- `word_to_coeff_repr`;
- the concrete instances `word_to_coeff_neg16505` and `word_to_coeff_26964`.

Mathematically, these lemmas do the indispensable bookkeeping which allows one to pass from signed machine integers to residue classes in $\mathbf{F}_q$. Cryptographically, this is the bridge by which a concrete read from memory may eventually be interpreted as a field element participating in a butterfly.

4 Explicit Twiddle Tables and Their Formal Role

4.1 Inverse Twiddle Table

The inverse zeta table is defined abstractly by

$$\text{zetas_inv}[k] = \text{zroot}^{-(\text{bsrevs}(k)+1)} \quad (0 \leq k \leq 254),$$

with the final entry replaced by

$$\text{scale255} = \text{inv}(256) = -252 \pmod{q}.$$

The following items are now fully formalized in `NTT_Fq.ec`:

- `unit_zroot`;
- `inv_zroot`;
- `zetas_invE`, giving the full explicit list of inverse twiddle factors;
- `scale255E`;
- `zetas_inv_vals`, expressing the Montgomery-scaled inverse table explicitly;
- `zetas_inv_255_montE`;

- `jzetas_inv_255_wordE`;
- `jzetas_inv_255_scaleE`.

The last two lemmas are especially significant. The extracted table `jzetas_inv` stores at index 255 not merely the abstract factor $\text{inv}(256)$, but the machine-level constant -29720 , representing the Montgomery-encoded quantity required by the Jasmin multiplication routine. The lemma

$$\text{jzetas_inv_255_scaleE}$$

establishes the exact compatibility relation

$$\text{word_to_coeff}(\text{jzetas_inv}[255]) \cdot R^{-1} = \text{scale255} \cdot R,$$

which is precisely the identity needed in the final scaling loop of the inverse transform.

4.2 Forward Twiddle Table

The forward table is treated similarly. The abstract definition

$$\text{zetas}[k] = \text{zroot}^{\text{bsrevs}(k)}$$

is matched by the explicit formulas

- `zetasE`;
- `zetas_vals`.

Moreover, the development already contains the index-1 bridge lemmas

- `jzetas_1_poly`;
- `jzetas_1_word_montE`;
- `jzetas_1_get32E`;
- `jzetas_1_get32dE`;
- `zetas_1_montE`;
- `jzetas_1E`;
- `jzetas_1_wordE`.

These serve as small but concrete witnesses that the forward table relation is mathematically the expected one. They also indicate the intended pattern for the full array-level proof.

5 Partial Reconstruction of the Twiddle-Bridge Proofs

The principal work currently in progress concerns the array-level bridge lemmas relating the extracted tables `jzetas` and `jzetas_inv` to the mathematical arrays `array256_mont zetas` and `array256_mont_inv zetas_inv`.

The present proof scripts introduce the stronger intermediate identities

```
jzetas_inv_polyE : barray256_to_poly Hpoly_extract.jzetas_inv = (array256_mont_inv zetas_inv)[255]
jzetas_polyE : barray256_to_poly Hpoly_extract.jzetas = (array256_mont zetas)[0 ← incoeff(0)].
```

From these one deduces the pointwise statements

$$\begin{aligned} \text{jzetas_inv_poly_vals } i & \quad (0 \leq i < 255), \\ \text{jzetas_poly_vals } i & \quad (1 \leq i < 256), \end{aligned}$$

and thence the direct get-lemmas `jzetas_inv_get` and `jzetas_get`.

Observation 1. *The special indices differ for conceptual reasons. For the inverse table, index 255 stores the Montgomery-encoded final normalization constant, not the plain array value `(array256_mont_inv zetas_inv)[255]`. For the forward table, index 0 is padding and is never read by the algorithm, because the counter is incremented before the first lookup.*

5.1 Current Obstruction

The mathematical structure of the proof is clear and essentially complete. The obstruction is local and syntactic. When one compiles `NTT_Fq.ec`, EasyCrypt currently fails in the proof of `jzetas_inv_polyE` at the line

```
rewrite BArray1024.get32_of_list32 1:/#.
```

The prover reports:

```
nothing to rewrite.
```

Thus the unresolved issue is not arithmetic but normalization of the concrete `BArray1024` accessor inside the full ambient theory. In isolated probe files the same rewrite shape is admissible, whereas inside the complete theory it fails to match the current goal term. This suggests a mismatch in the surface form of the accessor after unfolding `Hpoly_extract.jzetas` and `Hpoly_extract.jzetas_inv`, rather than any defect in the underlying mathematical argument.

6 Status of the Reconstruction

The present state of the formal development may be summarized as follows.

Component	Status	Mathematical significance
<code>ntt_spec_11</code>	proved	termination and losslessness of the forward imperative model

Component	Status	Mathematical significance
<code>invntt_spec_ll</code>	proved	termination and losslessness of the inverse imperative model
<code>exp_zroot_256</code>	proved	primitive-root structure and inverse scaling justification
<code>word_to_coeff</code> layer	proved	conversion from signed words to field coefficients
<code>poly_repr</code> layer	proved	base representation predicate for future simulation arguments
<code>zetas_invE</code> , <code>zetas_inv_vals</code>	proved	explicit inverse twiddle constants and Montgomery form
<code>zetasE</code> , <code>zetas_vals</code>	proved	explicit forward twiddle constants and Montgomery form
<code>jzetas_inv_255_scaleE</code>	proved	compatibility of the inverse final scaling constant
<code>jzetas_inv_polyE</code> and descendants	formalized but not compiling	inverse array bridge blocked by accessor normalization
<code>jzetas_polyE</code> and descendants	formalized but not compiling	forward array bridge blocked by the same normalization issue
<code>fqmul_corr</code>	not yet formalized	arithmetic correctness of one Montgomery multiplication
Butterfly correctness lemmas	not yet formalized	local simulation step for both transforms
Loop invariants for extracted Jasmin code	not yet formalized	pRHL refinement from extraction to imperative model
Imperative-to-abstract refinement	not yet formalized	identification of imperative NTT with <code>Rq.ntt</code> and <code>Rq.invntt</code>

7 Remaining Proof Obligations

Once the concrete table bridge is resolved, the remaining development is structurally straightforward, though not short.

7.1 Arithmetic Step

One must prove correctness of the extracted routine `M.__fqmul` using Montgomery reduction, relying on the existing theory `Fq.ec` and the lemma `SREDCp_corr` from `Montgomery.ec`. This yields the field-level interpretation of one machine multiplication.

7.2 Local Butterfly Lemmas

With the twiddle tables and multiplication routine in place, the next step is to establish:

- a forward butterfly correctness lemma;
- an inverse butterfly correctness lemma, including the final scaling phase.

These are the atomic transitions from which the loop invariants for the extracted procedures will be assembled.

7.3 Loop Refinement

The extracted routines `M._poly_ntt` and `M._poly_invntt` must then be related to the imperative procedures `NTT.ntt` and `NTT.invntt` by synchronized invariants on the indices `len`, `start`, `j`, and `zetasctr`. The natural logic here is probabilistic relational Hoare logic, though the programs are deterministic.

7.4 Abstract Identification

Finally, one must prove that the imperative models coincide with the abstract ring-level operators

$$\mathbf{Rq.ntt}, \quad \mathbf{Rq.invntt}.$$

This is where the Cooley–Tukey decomposition and the bit-reversal structure must be identified with the explicit formula from `Rq.ec`.

8 Recommended Next Step

The proof search should now focus narrowly on the accessor normalization problem in the twiddle-bridge lemmas. Concretely:

1. inspect the exact normal form of `BArray1024.get32 Hpoly_extract.jzetas_inv i` after unfolding `Hpoly_extract.jzetas_inv` inside the ambient theory;
2. identify the precise theorem name or rewriting direction accepted by the cloned `ByteArray` theory in this context;
3. only after that, resume the array extensional proofs of `jzetas_inv_polyE` and `jzetas_polyE`.

Remark 1. *At the present moment the main body of mathematical work is not blocked by a deep algebraic deficiency. The obstruction is a proof-engineering issue in the representation layer. This is good news: once the concrete memory accessor is brought under control, the path to the local butterfly lemmas is already visible.*

Conclusion

The proof effort has already secured the algebraic spine of the HAETAE NTT formalization. The imperative specifications are lossless; the chosen root is shown to have the required order; the explicit zeta tables, both forward and inverse, are mathematically identified; and the representation relation from word arrays to coefficient arrays is in place in its essential form. What remains is the completion of the concrete twiddle-table bridge and, beyond it, the expected sequence of simulation lemmas from Montgomery multiplication to butterflies, from butterflies to loops, and from loops to the abstract NTT operators. The present state is therefore best described not as incomplete in principle, but as suspended at a technically local juncture.